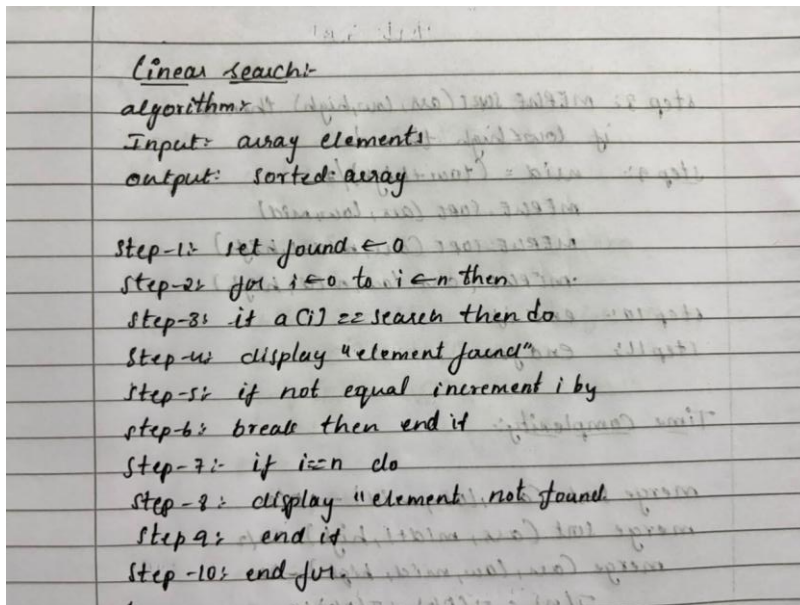


PRACTICAL-02

AIM: Implementation and Time analysis of linear and binary search algorithm.

1.) LINEAR SEARCH

ALGORITHM:



PROGRAM:

```
#include <iostream>

using namespace std;

int main() {
    int arr[20], n, key, i;

    cout << "Enter the number of elements: ";

    cin >> n;

    cout << "Enter the array elements: ";

    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    cout << "Enter the element to search: ";
```

```

cin >> key;

for (int i = 0; i < n; i++) {
    if (arr[i] == key) {
        cout << "Element found at index " << i;
        return 0;
    }
}

cout << "Element not found";

return 1;
}

```

OUTPUT:

```

PS C:\Users\MANCHEM HIMESH\OneDrive\Desktop\DA> cd "C:\Users\MANCHEM HIMESH\OneDrive\Desktop\DA" ; if ($?) { g++ linearsearch.cpp -o linearsearch } ; if ($?) {
{ ./linearsearch }
Enter the number of elements: 5
Enter the array elements: 5
12
23
34
45
Enter the element to search: 34
Element found at index 3
PS C:\Users\MANCHEM HIMESH\OneDrive\Desktop\DA>

```

TIME COMPLEXITY:

Linear search

Time complexity:-

for (i=0; i<n; i++)
 if (arr[i] == key)
 cout << "element found"
 found = 1;
 break;

$T(n) = 4n + 3$
 $\sim 4n$
 $\sim n$
 $T(n) = O(n)$

1. Best case = $O(1)$
 Ex: arr = {10, 20, 30, 40, 50}
 Key = 10, There is only one comparison
 $T(n) = 1$
 Time complexity = $O(1)$

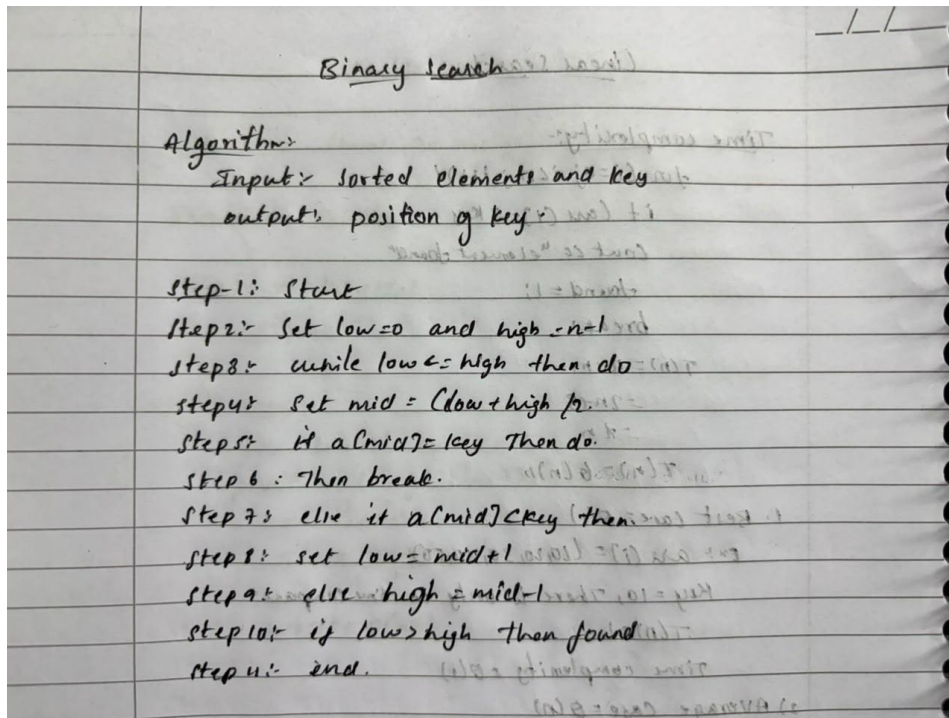
2) Average case = $O(n)$
 element found in middle
 ex: arr = {10, 20, 30, 40, 50}
 Key = 30 $\rightarrow$ 4 comparisons
 $T(n) = n/2$
 $T(n) = O(n)$

3) worst case = $O(n)$
 element found at last
 Ex: arr = {10, 20, 30, 40, 50}
 Key = 50, we check all element upto n
 $T(n) = n$
 $T(n) = O(n)$

Time complexity for linear search is $O(n)$

2.) BINARY SEARCH

ALGORITHM:



PROGRAM:

```
#include <iostream>

using namespace std;

int main()
{
    int n, key;
    int low, high, mid;

    cout << "Enter the number of elements: ";

    cin >> n;

    int a[n];

    cout << "Enter the sorted array elements: ";

    for (int i = 0; i < n; i++)
    {
        cin >> a[i];
    }

    cout << "Enter the key element: ";

    cin >> key;

    low = 0;
```

```

high = n - 1;
while (low <= high)
{
    mid = (low + high) / 2;
    if (a[mid] == key)
    {
        cout << "Element found at position " << mid << endl;
        break;
    }
    else if (a[mid] < key)
    {
        low = mid + 1;
    }
    else
    {
        high = mid - 1;
    }
}
if (low > high)
{
    cout << "Element not found" << endl;
}
return 0;
}

```

OUTPUT:

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
cd "c:\Users\MANCHEM HIMESH\OneDrive\Desktop\DAA\" ; if ($?) { g++ binarysearch.cpp -o binarysearch } ; if ($?) {
{ .\binarysearch }
Enter the number of elements: 5
Enter the sorted array elements: 21
32
45
55
67
Enter the key element: 67
Element found at position 4
PS C:\Users\MANCHEM HIMESH\OneDrive\Desktop\DAA>

```

TIME COMPLEXITY:

Time Complexity

```
low = 0; high = n-1; → 1
while (low <= high) { → log n
    mid = (low + high) / 2; → log n
    if (a[mid] == key) → log n
        break; → 1
    else if (a[mid] < key) → log n
        low = mid + 1; → log n
    else high = mid - 1; → log n
}
```

$T(n) = 1 + \log n + \log n + \log n + 1 + \log n + \log n + \log n$
 $= 2 + 6 \log n$
 $T(n) = O(\log n)$ - because in every iteration array divides into half.

1) Best case:- $O(1)$
 $mid = (low + high) / 2$
 $if (a[mid] == key)$
 $T(n) = 1 + 1 = 2$
 $T(n) = O(1)$

2) Average Case - $O(\log n)$
 $n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \dots$
 no. of divisions = $\log n$
 $T(n) = \log n$
 $T(n) = O(\log n)$

3) worst case - $O(\log n)$
 $n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \dots$
 $T(n) = \log n$
 $T(n) = O(\log n)$

Conclusion –

Linear Search

Linear Search is a simple searching technique that checks each element one by one until the required element is found. It is easy to implement and works on both sorted and unsorted arrays, but its time complexity is $O(n)$.

Binary Search

Binary Search is an efficient searching technique that repeatedly divides a sorted array into two halves to locate the required element. It is faster than linear search, with a time complexity of $O(\log n)$, but it requires the data to be sorted.